Project 3: Motor Control

ELEC 553: Mobile and Embedded Systems
Rice University

Spring 2026

Team Member: Ashley Garcia, Mehul Goel, Valeria Itrat Herrera, Sugiarto Wibowo

Project 3: Motor Control

ELEC 553 Mobile and Embedded Systems

Team Name: The Owl-tonomous Aviators
Team Member: Ashley Garcia, Mehul Goel, Valeria Itrat Herrera, Sugiarto Wibowo

The goal of this project is to set up the Raspberry Pi 5 and the provided hardware to build the RC car (Figure 1), and to run a Python script that makes the car follow the sequence: move forward, stop, turn left, and turn right. We started by setting up the Raspberry Pi with a customized OS from Project 3. Then, we connected to the RPi wirelessly via SSH. Next, we wired the ESC board to RPi GPIO 18 to control the motor speed via PWM, and to GPIO 19 to control the steering via PWM. With all the components connected, we calibrated the motor speed with the ESC board by following the steps provided in the project, including:

1. Hold the button and switch the ESC power on, then wait until the device beeps.
2. Set GPIO 18 (PWM speed) to output a 7.5% duty cycle at 50 Hz.
3. Press and release the button; one beep occurs.
4. Change the duty cycle to 10% at 50 Hz.
5. Press and release the button; two beeps occur.
6. Change the duty cycle to 5% at 50 Hz.
7. Press and release the button; three beeps occur.
8. Turn the switch off.

After calibration, the motor can be used. Every time the program starts, the motor needs to be initialized with a 7.5% duty cycle at 50 Hz, and this can be changed afterward (Figure 2). The robot's steering is also controlled by PWM. We determined the direction based on the PWM output as follows:

- 6.0% at 50Hz to turn left
- 7.5% at 50Hz to move forward
- 9.0% at 50Hz to turn right

Finally, the robot was programmed to move following a specific sequence: forward, stop, left, right, and straight (Figure 3).

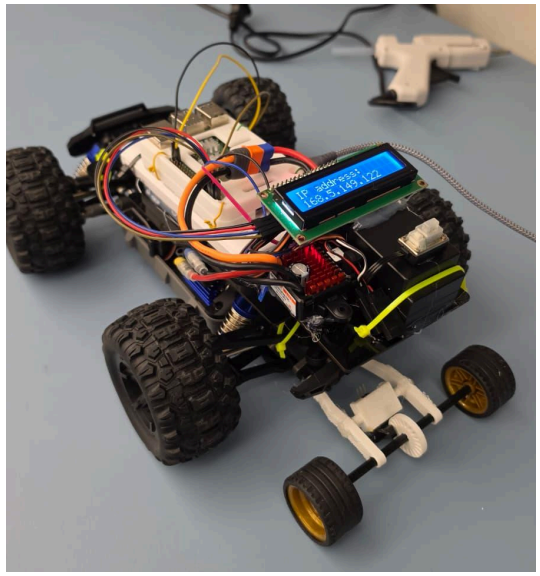

Figure 1: Hardware setup

```python
# Minimum DC PWM
default_speed_pwm = 8.05
steering_forward = 7.5
steering_right = 9.0
steering_left = 6.0


class MotorController:
    def __init__(self, speed_pin=18, steering_pin=19, _steering_direction=steering_forward):
        self.speed_pin = speed_pin
        self.steering_pin = steering_pin

        GPIO.setmode(GPIO.BCM)

        GPIO.setup(self.speed_pin, GPIO.OUT)
        self.speed_pwm = GPIO.PWM(self.speed_pin, 50) # set the frequency to 50 Hz
        self.speed_pwm.start(7.5) # Don't change the 7.5 DC because it is for initialization

        GPIO.setup(self.steering_pin, GPIO.OUT)
        self.steering_pwm = GPIO.PWM(self.steering_pin, 50) # set the frequency to 50 Hz
        self.steering_pwm.start(_steering_direction) # Start with the specified duty cycle

    def set_speed(self, duty_cycle):
        self.speed_pwm.ChangeDutyCycle(duty_cycle)

    def set_steering(self, direction="forward"):
        if direction == "forward":
            self.steering_pwm.ChangeDutyCycle(steering_forward)
        elif direction == "right":
            self.steering_pwm.ChangeDutyCycle(steering_right)
        elif direction == "left":
            self.steering_pwm.ChangeDutyCycle(steering_left)

    def stop(self):
        self.set_speed(0)
        self.speed_pwm.stop()
        self.steering_pwm.stop()
        GPIO.cleanup()
```

Figure 2: Motor Controller setup

```python
if __name__ == "__main__":
    motor_controller = MotorController() # Initialize motor controller with default pins and steering direction

    try:
        while True:
            # Move forward
            motor_controller.set_steering("forward")
            motor_controller.set_speed(default_speed_pwm)
            sleep(0.5)

            # Stop
            motor_controller.set_speed(0)
            sleep(0.5)

            # Turn left
            motor_controller.set_steering("left")
            motor_controller.set_speed(default_speed_pwm-0.1) # Set speed back to default
            sleep(1)

            # Turn right
            motor_controller.set_steering("right")
            motor_controller.set_speed(default_speed_pwm-0.1) # Set speed back to default
            sleep(1)

            # Turn straight
            motor_controller.set_steering("left")
            motor_controller.set_speed(default_speed_pwm-0.1) # Set speed back to default
            sleep(1)
    except KeyboardInterrupt:
        pass
```

Figure 3: Operation sequence

One challenge during project development was that the motor must be initialized with a 7.5% duty cycle whenever it boots up, or it might not perform as expected. Another issue during setup was the battery level; if it was not fully charged, it also affected performance. To improve hardware installation and avoid potential electrical issues, we 3D-printed a case for the Raspberry Pi to protect it. Another interesting application of this approach would be to incorporate a camera to detect the environment and the road, allowing the car to move automatically along a specific path.

AI Acknowledgement

I acknowledge the use of ChatGPT-5.2 and Grammarly to check English spelling and grammar and ensure the report's writing was clear. We asked ChatGPT not to change the main ideas or add any new information.

References

- User raja_961, "Autonomous Lane-Keeping Car Using Raspberry Pi and OpenCV". Instructables. URL: [Autonomous Lane-Keeping Car Using Raspberry Pi and OpenCV : 7 Steps (with Pictures) - Instructables]